Set up your lab: SRE for AI-ML Systems

When you finish, you will have a small model service running on Linux that you can measure, break and fix, ready for Modules 1 to 3. It takes about 10 to 15 minutes, or less with Codespaces.

What you need

- A computer that can run Linux in one of three ways: GitHub Codespaces in your browser, WSL Ubuntu on Windows, or a Multipass Ubuntu VM on a Mac. The lab checks that it is running on Linux.
- A free GitHub account, if you use Codespaces.
- Python 3, git and curl. The lab uses only the standard library, so you do not need `pip install`.
- Port 8080 free. The model service listens on it.
- About 50 MB of free disk.
- **No GPU, no Docker, no API keys and no internet** once you have the files.

Step 1 · Pick where to run it

Option A: GitHub Codespaces (easiest, any computer)

1. Sign in to github.com and open this repo.
2. Click **Code > Codespaces > Create codespace on main**.
3. Wait for the terminal at the bottom. Python 3, git and curl are already installed.

Option B: Windows (WSL / Ubuntu)

1. Open **PowerShell as Administrator** and run:

```
wsl --install -d Ubuntu-24.04
```

1. Restart your computer, then open **Ubuntu** from the Start menu and create a Linux user.
2. Install the tools:

```
sudo apt update
sudo apt install -y git curl unzip python3
```

Option C: Mac (Multipass VM)

1. Install Homebrew first (https://brew.sh), then Multipass:

```
brew install --cask multipass
multipass launch 24.04 --name ailab
multipass shell ailab
```

1. Inside the VM, install the tools:

```
sudo apt update
sudo apt install -y git curl unzip python3
```

Step 2 · Get the lab files

Download sre-for-ai-ml-lab.zip from the `downloads/` folder of this repo. Then unzip it and go into the folder:

```
unzip sre-for-ai-ml-lab.zip
cd sre-for-ai-ml
```

On a Mac with Multipass, copy the zip into the VM first. Run this on your Mac, not inside the VM:

```
multipass transfer ~/Downloads/sre-for-ai-ml-lab.zip ailab:
```

Step 3 · Install and check

You do not need to install anything else. Run the health check:

```
bash setup/doctor.sh
```

You should see one `[ OK ]` line each for Linux, Python 3, git, curl and "Port 8080 is free", and then `No GPU or Docker needed for this course.` If a line says `[WARN]`, the fix is printed on the same line.

Step 4 · Quick test for each module

Every module starts with its own `start.sh`. It gives you a fresh copy of the service in `~/triage-sre`, with model v1 and empty logs. You can run it again any time your lab gets into a mess.

Module 1 · Measure What Users Feel

```
bash modules/module-1/start.sh && cd ~/triage-sre
python3 server.py &
curl -s "localhost:8080/predict?text=payment+failed"
python3 load.py --requests 300 && python3 sli.py
```

You should see `ticket-triage serving model v1 on :8080`, then `{"label": "urgent", "confidence": 0.9, "model": "v1"}`, then `sent 300 requests: 300 ok, 0 failed`, three `PASS` lines and `SLOs MET`. Stop the server with `kill $(cat server.pid)`.

Module 2 · Spend the Error Budget (run from the `sre-for-ai-ml` folder)

```
bash modules/module-2/start.sh && cd ~/triage-sre
python3 simulate.py quiet && python3 budget.py
```

You should see `wrote 30 days (43,200 minutes) of 'quiet' traffic to history.csv`, then the budget lines, and the result ends with `OK no alert. Blips happen; the budget absorbs them.`

Module 3 · When It Breaks (run from the `sre-for-ai-ml` folder)

```
bash modules/module-3/start.sh && cd ~/triage-sre
python3 server.py &
bash .lab/incident.sh 1
python3 sli.py --last 300
```

You should see `PAGER: ticket-triage SLO alert fired. You are on call. Open RUNBOOK.md.`, and then `SLO BREACH`. You will fix this in the live session. To reset everything, run `start.sh` again.

If something goes wrong

- **`[WARN] Not Linux`**: you are on plain macOS or Windows. Use Codespaces, WSL or Multipass (Step 1).
- **`[WARN] Port 8080 is busy`**: stop the other program, or start the server with `PORT=8081 python3 server.py &` and add `--port 8081` to `load.py`.
- **`python3`, `git` or `curl` missing**: run `sudo apt install -y python3 git curl`, as the doctor suggests.
- **`no requests logged yet - run load.py first`**: the server has not answered anything yet. Start `server.py`, then run `load.py`.
- **Numbers look strange after a change**: old traffic is mixed in. Use `python3 sli.py --last 300`, or run `start.sh` for a clean copy.

- **Stuck for more than 15 minutes on WSL or Multipass?** Switch to Codespaces. It works the same way in any browser.

Official guides

- GitHub Codespaces: https://docs.github.com/codespaces
- Install WSL: https://learn.microsoft.com/windows/wsl/install
- Install Multipass: https://canonical.com/multipass/install
- Homebrew: https://brew.sh
- Python downloads: https://www.python.org/downloads/